CLIPP — Mixed-Integer Linear Programming Formulation

1 Problem overview

The **Climate-Integrated Street Cleaning Routing Problem (CLIPP)** plans routes for a heterogeneous fleet of cleaning vehicles on a directed street network. The objective is to maximise the total length of streets cleaned while minimising water waste, subject to a shared time budget and mandatory-coverage requirements.

2 Notation

2.1 Sets

Symbol	Description
$\mathcal{N} = \{0, \dots, N-1\}$	Junctions (nodes)
$\mathcal{J} = \{0, \dots, M-1\}$	Streets (edges)
$\mathcal{J}_M \subset \mathcal{J}$	Mandatory streets
$\mathcal{J}_O \subset \mathcal{J}$	Optional streets
$\mathcal{J}_C \subset \mathcal{J}$	Connector streets ($\mathcal{J} = \mathcal{J}_M \cup \mathcal{J}_O \cup \mathcal{J}_C$)
$\mathcal{A}$	Directed arcs (a two-way street generates two arcs)
$A_j \subseteq \mathcal{A}$	Set of arcs corresponding to street j
$\mathcal{K} = \{0, \dots, C-1\}$	Vehicles
$\delta^+(u), \delta^-(u)$	Outgoing / incoming arcs at node u

2.2 Parameters

Symbol	Description
T	Shared time limit for every vehicle (seconds)
S	Depot junction
W	Water-waste penalty factor
c_a	Traversal time of arc a (seconds)
ℓ_j	Length of street j (metres)
r_j	Cleaning requirement of street j (litres / km)
q_k	Water capacity of vehicle k (litres / km)
U_a	Traversal upper bound for arc a: $U_a = \min\big(\lfloor T/c_a \rfloor,\ 50\big)$
$\mathcal{M}$	Big-M constant: $\mathcal{M} = 1 + \sum_{a \in \mathcal{A}} U_a$

3 Decision variables

$$f_{a,k} \in \mathbb{Z}_{\geq 0} \qquad \forall\, a \in \mathcal{A},\ k \in \mathcal{K} \qquad \text{(flow variable)}$$

$$x_{j,k} \in \{0,1\} \qquad \forall\, j \in \mathcal{J} \setminus \mathcal{J}_C,\ k \in \mathcal{K} \qquad \text{(cleaning assignment)}$$

$$y_j \in \{0,1\} \qquad \forall\, j \in \mathcal{J} \setminus \mathcal{J}_C \qquad \text{(cleaned indicator)}$$

- $f_{a,k}$ counts how many times vehicle k traverses arc a.
- $x_{j,k} = 1$ iff vehicle k cleans street j.
- $y_j = 1$ iff street j is cleaned by *any* vehicle.

Note. If vehicle k cannot clean street j (i.e. $q_k < r_j$, or $j \in \mathcal{J}_C$), $x_{j,k}$ is fixed to 0 and not created as a decision variable.

Implementation (OR-Tools CP-SAT).

```python
# f[a, k]: number of times vehicle k traverses arc a
def _create_flow_vars(model, A, C, arc_max):
    f = {}
    for k in range(C):
        for a in range(A):
            f[a, k] = model.new_int_var(0, arc_max[a], f"f_{a}_{k}")
    return f

# clean[j, k]: BoolVar if vehicle k can clean street j, else fixed 0
def _create_clean_vars(model, M, C, streets, vehicles):
    clean = {}
    for j in range(M):
        s = streets[j]
        for k in range(C):
            if s["cat"] != "C" and vehicles[k]["cap"] >= s["req"]:
                clean[j, k] = model.new_bool_var(f"clean_{j}_{k}")
            else:
                clean[j, k] = 0  # cannot clean -> not a variable
    return clean

# cleaned[j]: BoolVar for non-connector streets
def _create_cleaned_vars(model, M, streets):
    cleaned = {}
    for j in range(M):
        if streets[j]["cat"] != "C":
            cleaned[j] = model.new_bool_var(f"cleaned_{j}")
        else:
            cleaned[j] = 0
    return cleaned
```

Listing 1: Decision variable creation (`clipp/model.py`)

4 Constraints

4.1 Flow conservation

$$\sum_{a \in \delta^+(u)} f_{a,k} \;=\; \sum_{a \in \delta^-(u)} f_{a,k} \qquad \forall\, u \in \mathcal{N},\ k \in \mathcal{K} \tag{1}$$

Each vehicle's route forms a *closed walk*: the total flow leaving every junction equals the total flow entering it. This guarantees that the vehicle can always return to its starting point (the depot) without "teleporting".

```python
def _add_flow_conservation(model, f, N, C, out_arcs, in_arcs):
    for k in range(C):
        for u in range(N):
            out_f = [f[a, k] for a in out_arcs.get(u, [])]
            in_f  = [f[a, k] for a in in_arcs.get(u, [])]
            if out_f or in_f:
                model.add(sum(out_f) == sum(in_f))
```

Listing 2: Constraint (1): Flow conservation

4.2 Time limit

$$\sum_{a \in \mathcal{A}} c_a \, f_{a,k} \;\leq\; T \qquad \forall\, k \in \mathcal{K} \tag{2}$$

The total traversal time of every vehicle must not exceed the shared time budget T.

```python
def _add_time_constraints(model, f, A, C, T, arc_cost):
    for k in range(C):
        model.add(sum(arc_cost[a] * f[a, k] for a in range(A)) <= T)
```

Listing 3: Constraint (2): Time limit

4.3 Traversal requirement for cleaning

$$\sum_{a \in A_j} f_{a,k} \;\geq\; x_{j,k} \qquad \forall\, j \in \mathcal{J} \setminus \mathcal{J}_C,\; k \in \mathcal{K} \tag{3}$$

If vehicle k cleans street j ($x_{j,k} = 1$), it must have traversed at least one arc of that street. A vehicle cannot clean a street it has not visited.

```python
def _add_traversal_requirements(model, f, clean, M, C, streets, street_arcs):
    for j in range(M):
        if streets[j]["cat"] == "C":
            continue
        for k in range(C):
            if not isinstance(clean[j, k], int):  # skip fixed-zero vars
                traversals = [f[a, k] for a in street_arcs[j]]
                if traversals:
                    model.add(sum(traversals) >= clean[j, k])
```

Listing 4: Constraint (3): Traversal requirement

4.4 Cleaned-indicator linkage

$$x_{j,k} \;\Rightarrow\; y_j \qquad \forall\, j \in \mathcal{J} \setminus \mathcal{J}_C,\; k \in \mathcal{K} \tag{4}$$

$$y_j \;\Rightarrow\; \bigvee_{k \in \mathcal{K}} x_{j,k} \qquad \forall\, j \in \mathcal{J} \setminus \mathcal{J}_C \tag{5}$$

Together these encode the logical equivalence $y_j \Leftrightarrow \bigvee_k x_{j,k}$. Constraint (4) ensures that the cleaned indicator is activated whenever any vehicle cleans the street. Constraint (5) prevents $y_j = 1$ unless at least one vehicle actually cleans street j.

```python
def _add_cleaned_linkage(model, clean, cleaned, M, C, streets):
    for j in range(M):
        if streets[j]["cat"] == "C":
            continue
        bool_vars = [
            clean[j, k] for k in range(C)
            if isinstance(clean[j, k], cp_model.IntVar)
        ]
        if bool_vars:
            for bv in bool_vars:
                model.add_implication(bv, cleaned[j])        # x_{j,k} => y_j
            model.add_bool_or(bool_vars + [cleaned[j].Not()])# y_j => OR_k x_{j,k}
        else:
            model.add(cleaned[j] == 0)
```

Listing 5: Constraints (4) – (5): Cleaned linkage

4.5 Single-vehicle cleaning

$$\sum_{k \in \mathcal{K}} x_{j,k} \leq 1 \qquad \forall j \in \mathcal{J} \setminus \mathcal{J}_C \tag{6}$$

Each street is cleaned by *at most one* vehicle. While the problem statement allows redundant cleaning, the solver enforces single-vehicle cleaning to avoid unnecessary water waste (the street's length is scored only once regardless).

```python
def _add_single_cleaner(model, clean, M, C, streets):
    for j in range(M):
        if streets[j]["cat"] == "C":
            continue
        bool_vars = [
            clean[j, k] for k in range(C)
            if isinstance(clean[j, k], cp_model.IntVar)
        ]
        if len(bool_vars) > 1:
            model.add(sum(bool_vars) <= 1)
```

Listing 6: Constraint (6): Single-vehicle cleaning

4.6 Mandatory coverage

$$y_j = 1 \qquad \forall j \in \mathcal{J}_M \tag{7}$$

Every mandatory street must be cleaned. A solution that violates this constraint is infeasible.

Relaxation. If a mandatory street is provably unreachable within the time budget (detected via a Dijkstra-based pre-check), its hard constraint is relaxed to a soft one — it is still scored if cleaned, but the solver will not be forced into infeasibility.

```python
def _add_mandatory_constraints(model, cleaned, M, streets, relaxed_mandatory):
    for j in range(M):
        if streets[j]["cat"] == "M":
```

```
            if j in relaxed_mandatory:
                continue  # provably unreachable -- keep as soft
            assert isinstance(cleaned[j], cp_model.IntVar)
            model.add(cleaned[j] == 1)
```

Listing 7: Constraint (7): Mandatory coverage

4.7 Depot inclusion (Big-M)

$$\sum_{a \in \mathcal{A}} f_{a,k} \;\leq\; \mathcal{M} \cdot \sum_{a \in \delta^+(S)} f_{a,k} \qquad \forall\, k \in \mathcal{K} \tag{8}$$

If vehicle k uses any arc at all (left-hand side > 0), then it must have at least one outgoing traversal from the depot. Combined with flow conservation (1), this forces the vehicle's closed walk to pass through the depot. When the depot has zero outgoing flow, the right-hand side becomes 0, prohibiting any arc usage — i.e. an unused vehicle simply stays at the depot.

```
def _add_depot_inclusion(model, f, A, C, S, out_arcs, arc_max):
    big_M_val = sum(arc_max) + 1
    for k in range(C):
        depot_out_arcs = out_arcs.get(S, [])
        if depot_out_arcs:
            depot_out_sum  = sum(f[a, k] for a in depot_out_arcs)
            total_flow_sum = sum(f[a, k] for a in range(A))
            model.add(total_flow_sum <= big_M_val * depot_out_sum)
```

Listing 8: Constraint (8): Depot inclusion (Big-M)

5 Objective function

$$\max \quad \underbrace{\sum_{j \in \mathcal{J} \setminus \mathcal{J}_C} \ell_j \, y_j}_{\text{total length cleaned}} \;-\; W \cdot \underbrace{\sum_{j \in \mathcal{J} \setminus \mathcal{J}_C} \sum_{k \in \mathcal{K}} \frac{(q_k - r_j)\,\ell_j}{1000} \, x_{j,k}}_{\text{total water wasted (litres)}} \tag{9}$$

- The first term rewards cleaning longer streets.
- The second term penalises water waste: for every street cleaned, the *excess* capacity $(q_k - r_j)$ in litres / km times the street length (converted from metres to kilometres by dividing by 1000) gives the wasted litres.
- The penalty factor W controls the trade-off between coverage and efficiency. When $W = 0$ the problem reduces to pure coverage maximisation.

Implementation detail. To keep the solver in integer arithmetic the entire objective is scaled by a factor of 1000; this does not change the optimal solution.

```
_OBJ_SCALE = 1000

def _set_objective(model, clean, cleaned, M, C, streets, vehicles, W):
    W_scaled = int(round(W * _OBJ_SCALE))
    obj_terms = []

    for j in range(M):
        s = streets[j]
        if s["cat"] == "C":
```

```
            continue

        # Positive term: length contribution
        if isinstance(cleaned[j], cp_model.IntVar):
            obj_terms.append(_OBJ_SCALE * s["length"] * cleaned[j])

        # Negative term: water-waste penalty
        for k in range(C):
            if not isinstance(clean[j, k], cp_model.IntVar):
                continue
            excess = vehicles[k]["cap"] - s["req"]
            if excess > 0 and W_scaled > 0:
                penalty = W_scaled * excess * s["length"] // 1000
                if penalty > 0:
                    obj_terms.append(-penalty * clean[j, k])

    model.maximize(sum(obj_terms) if obj_terms else 0)
```

Listing 9: Objective function implementation

6 Subtour elimination (iterative)

Flow conservation alone does not prevent *subtours* — disconnected cycles that never visit the depot. These are eliminated lazily via an iterative callback:

1. Solve the model.
2. For each vehicle k, compute the weakly-connected components of its arc-flow multigraph.
3. For every component $\mathcal{C}$ that does *not* contain the depot, add the cut:

$$\sum_{\substack{a\in\mathcal{A}:\\ \text{head}(a)\in\mathcal{C}\\ \text{tail}(a)\in\mathcal{C}}} f_{a,k} \;\le\; \left(1 + \sum_{\substack{a\in\mathcal{A}:\\ \text{head}(a)\in\mathcal{C}\\ \text{tail}(a)\in\mathcal{C}}} U_a\right) \cdot \sum_{\substack{a\in\mathcal{A}:\\ \text{tail}(a)\notin\mathcal{C}\\ \text{head}(a)\in\mathcal{C}}} f_{a,k} \tag{10}$$

   If no arcs enter the component, all internal arcs are set to 0.
4. Re-solve with the new constraints (warm-started).
5. Repeat until no subtours remain or the time budget is exhausted.

```
def _eliminate_subtours(model, f, f_vals, arcs, arc_max, A, C, S):
    subtours_found = False

    for k in range(C):
        total_used = sum(f_vals[a, k] for a in range(A))
        if total_used == 0:
            continue

        adj_k = build_adj_multigraph(k, f_vals, arcs)
        comps = connected_components(adj_k)

        for comp in comps:
            if S in comp:
                continue  # connected to depot -- OK

            internal = [
                a for a in range(A)
                if arcs[a][0] in comp and arcs[a][1] in comp
```

```
            ]
            entering = [
                a for a in range(A)
                if arcs[a][0] not in comp and arcs[a][1] in comp
            ]
            if not internal:
                continue

            if entering:
                big_M_sub = sum(arc_max[a] for a in internal) + 1
                model.add(
                    sum(f[a, k] for a in internal)
                    <= big_M_sub * sum(f[a, k] for a in entering)
                )
            else:
                for a in internal:
                    model.add(f[a, k] == 0)

            subtours_found = True
    return subtours_found
```

Listing 10: Subtour elimination implementation

7 Post-processing: route reconstruction

After solving, each vehicle's arc-flow solution is converted into an explicit junction sequence via **Hierholzer's algorithm** for Eulerian circuits. If any connected component of a vehicle's multigraph is disconnected from the depot, a shortest-path bridge (depot $\rightarrow$ component $\rightarrow$ depot) is inserted when the remaining time budget permits; otherwise the component's arcs are dropped.